

FPGA-based acceleration of QC-LDPC syndrome encoding for QKD systems

Marco Origlia*, Maximilien Pinaud[‡], Luca Maggiani[‡], Piero Castoldi* and Nicola Andriolli[†]

*TeCIP Institute, Sant'Anna School of Advanced Studies, Pisa IT

[‡]Sma-RTy Italia SRL, Carugate IT

[†]Dep. of Information Engineering, University of Pisa, Pisa IT

Abstract—Quantum Key Distribution (QKD) systems will be fundamental in ensuring secure communications in future networks. However, due to device imperfections and the random nature of quantum physics itself, these systems need an information reconciliation phase to correct for errors in the key exchange. In particular, for Continuous Variable QKD (CV-QKD) systems, it is possible to employ Multi-Dimensional Reconciliation (MDR) with Quasi-Cyclic LDPC (QC-LDPC) error correction codes. To achieve good reconciliation with low SNR or to limit finite-size effects, large input block codes should be employed. FPGA acceleration is here envisioned. In this work we implemented the encoding for syndromes as large as 1Mbit with a QC-LDPC lift factor of 512 bits and achieve a throughput exceeding 3.55 Gb/s.

I. INTRODUCTION

Although rapid development of quantum computers will open the floor to much more powerful computations, this will also threaten the current cryptographic infrastructure, based on a computational barrier to the solution of NP-hard problems, such as discrete logarithms and large number factorization. In fact, Shor's quantum algorithm [1] reduces their computational complexity to polynomial. To safeguard future communications, Quantum Key Distribution (QKD) has been extensively studied and commercial deployments are currently available.

The aim of QKD is to create a shared secret among two remote parties over a link. Secrecy is guaranteed by the fundamental laws of physics.

In the asymptotic regime, i.e., for infinitely long data blocks, both security against collective attacks and especially optimality of such attacks has been proven, but for obvious practical reasons, QKD systems must operate on finite-size data blocks and this must be taken into account when calculating the secret key rate. In fact, despite optimality of Gaussian attacks among collective attacks has been proven for the finite-size regime and a Gaussian modulation [2], collective attacks may not be optimal. Therefore, a penalty must be introduced when calculating the practical key rate in such scenarios and the size of the data blocks plays an very relevant role [3]. The ability to process large amount of data is therefore vital for increasing the link distance or to operate at low SNR conditions. This aspect sets system requirements especially on the data post-processing phases, i.e. information reconciliation and privacy amplification. Concerning reconciliation, this implies that very large syndrome blocks must be computed. For this purpose, Quasi-Cyclic LDPC (QC-LDPC) codes may be employed. In

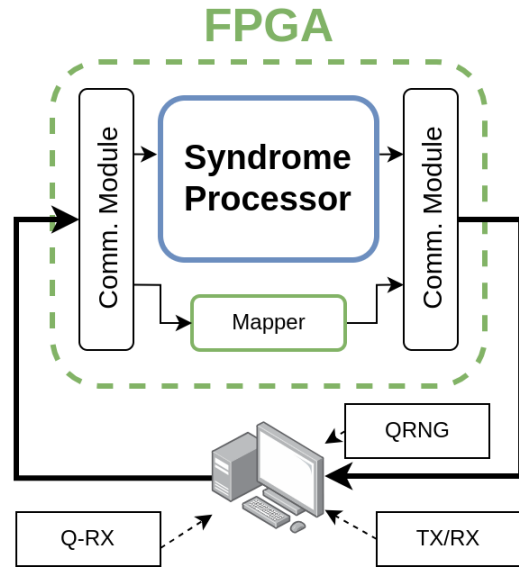


Fig. 1. System overview

the reconciliation case, the LDPC encoders have to evaluate the syndrome of a bit sequence, and decoders will possibly correct for errors in a copy of the same bit sequence by trying to match its parity to the syndrome. Due to intensive computation requirements, post-processing operations can benefit from hardware implementations [4]. Different solutions have been proposed, for both encoding and decoding tasks, in this direction: both GPUs [5] and FPGAs [6]–[8] have been used to process QC-LDPC codes in Multi-Dimensional Reconciliation (MDR).

In this work we focus on the design of a Syndrome encoder for Information Reconciliation, addressing 1Mb and larger input words. We show its feasibility, assess the amount of resources for its FPGA implementation and its throughput.

II. ARCHITECTURE

Fig. 1 sketches an overview of the designed syndrome encoder for QC-LDPC codes. A QKD terminal, connected to a quantum receiver (QRX), a classical transceiver (TX/RX) and a random data source (such as a Quantum Random Number Generator, QRNG), offloads computation to the FPGA and retrieves the results. Custom communication modules manage

data transfer to and from the external FPGA. In case of MDR, the FPGA device hosts a Syndrome Processor and a Mapper. The Syndrome Processor is detailed in Fig. 2. It takes an input word $\mathbf{u} \in \mathbb{F}_2^n$ and outputs the corresponding syndrome $\mathbf{s} \in \mathbb{F}_2^m$. The syndrome is derived according to a parity matrix $\mathbf{H} \in \mathbb{F}_2^{m \times n}$ as $\mathbf{s} = \mathbf{H} \cdot \mathbf{u}$, where products and sums are in the Galois Field $\mathcal{GF}(2)$. Being the code Quasi-Cyclic, the matrix has a block structure with all square sub-matrices $\mathbf{H}_{i,j} \in \mathbb{F}_2^{z \times z}$ of size z , the lift factor of the code. $\mathbf{H}_{i,j}$ are either null- or rotation-matrices, and they can be described by a set of rotation values $r_{i,j}$, defined wherever the $\mathbf{H}_{i,j} \neq \mathbf{0}$. For the sake of processing, both $\mathbf{s}$ and $\mathbf{u}$ can be seen as made up of multiple z -sized blocks $\mathbf{s}_i$ and $\mathbf{u}_j$ respectively. They are linked by the relation $\mathbf{s}_i = \sum_{j=1}^N \mathbf{H}_{i,j} \mathbf{u}_j$. In the Data Loading section of the module, input data are synchronised with parity matrix coefficients stored in an on-chip ROM. Data is then delivered to the Rotator in charge of processing the data set $(\mathbf{u}_j, \{r_{i,j} \mid i \in \mathbb{N}_0, i \leq M, \mathbf{H}_{i,j} \neq \mathbf{0}\})$. A Block Distributor delivers the rotated data at the output of Rotators to the Syndrome Updaters, which are responsible for managing a specific memory segment where the partial syndrome is stored. The syndrome is in fact split in multiple segments to allow concurrent update from multiple Syndrome Updaters. Partial syndrome blocks are updated according to the iterative formula $\mathbf{s}_i^k = \mathbf{s}_i^{k-1} + \mathbf{H}_{i,k} \cdot \mathbf{u}_k$, with $\mathbf{s}_i^0 = \mathbf{0}$, until completion of processing of the last input block, to obtain the final syndrome block $\mathbf{s}_i = \mathbf{s}_i^N$.

When the Block Distributor has received and delivered the last rotated block, it gives notice to the Syndrome Collector that the syndrome is complete. Finally the Syndrome Collector will coordinate syndrome block output by collecting data from Syndrome Downloaders. The following subsections describe Rotators, the core module of the Syndrome Processor, and how to store the parity matrix in the on-chip memory.

A. Rotator design principle

Some input blocks affect multiple syndrome blocks, i.e. the product $\mathbf{H}_{i,j} \cdot \mathbf{u}_j$ is not identically null for at least 2 syndrome indices $i_1, \dots, i_{M_j}$. To avoid carrying out the same rotation multiple times for the same input blocks, we designed a rotator to perform incremental rotations, so they accept one input block and a sequence of rotation values, so if $r_{i_1,j} < \dots < r_{i_k,j}$ are sorted, the module first rotates $\mathbf{u}_j$ by $r_{i_1,j}$, and outputs the result (denoted as $\mathbf{u}_j^{r_{i_1,j}} = R_{r_{i_1,j}}(\mathbf{u}_j)$) to the downstream, where partial syndrome block $\mathbf{s}_{i_1}^{j-1}$ will be updated to $\mathbf{s}_{i_1}^j = \mathbf{s}_{i_1}^{j-1} + \mathbf{u}_j^{r_{i_1,j}}$. Subsequently a further rotation is applied to the input block, but this time, instead of applying the full rotation, we can apply the residual rotation $r_{i_2,j} - r_{i_1,j}$ to the previous output and we obtain $\mathbf{u}_j^{r_{i_2,j}} = R_{r_{i_2,j}-r_{i_1,j}}(\mathbf{u}_j^{r_{i_1,j}})$. This operation can be repeated until the end of the rotation values sequence is reached.

B. Storage of the parity check matrix

To avoid the need to store input blocks $\mathbf{u}_j$, we constrain the design to process all partial products $\mathbf{H}_{i,j} \cdot \mathbf{u}_j$ back to back. This implies that the elements of $\mathbf{H}$ should be stored

in columns for easier memory access. Exploiting sparsity of $\mathbf{H}$, we can actually store just the rotation values $r_{i,j}$. Moreover we avoid storing empty locations, corresponding to null $\mathbf{H}_{i,j}$ matrices. To preserve the information of the row i corresponding to the current $r_{i,j}$, we simply store i in the same location as the corresponding rotation value. In order to work with irregular codes, where the number of non null matrices per block column is not constant, we should add further matrix control data in the same memory location, for instance a last-of-column flag F_l and a bottom-of-matrix flag F_b to the memory locations. In summary, each memory location contains a tuple $(F_b, F_l, i, r_{i,j})$. F_l will be used by Rotators (along with $r_{i,j}$) to know when current-block processing is complete and they can accept the next input block $\mathbf{u}_j$. The Rotation Value Loader will use both F_l and F_b values. F_l is used to know when the next sequence of locations will have to be directed to a different Rotator. F_b is used to know when to wait for the downstream to update the last syndrome blocks before restarting from the initial location. i will be passed over by the Rotators to let the Syndrome Updaters know which syndrome block to update with the rotated block $\mathbf{u}_j^{r_{i,j}}$.

III. IMPLEMENTATION AND RESULTS

The hardware description of the Syndrome Processor has been written in Verilog and it has been tested on a Multi-Edge-Type QC-LDPC code with lift factor $z = 512$, a syndrome size $n = 1024$ kbit and a rate R of 0.02. Operation of every module has been extensively simulated and their output has been checked using Icarus Verilog and, when integration with Intel's IPs was needed or convenient, Modelsim. The whole Syndrome Processor was then tested on a sequence of 10 input words. The output sequence matched the expected result sequence, pre-evaluated with a Python script, making use of the `galois` library module. Simulations allow to count how many clock cycles occur between two consecutive available syndromes. Maximum clock cycle count T_c can be obtained when input data are always valid and the downstream is always ready to withdraw outputs. Given a target operating frequency $f=125$ MHz, the corresponding throughput is $\theta = \frac{m \cdot f}{T_c}$ [bit/s]. Finally, maximum operating frequency is assessed to exceed 125 MHz by Intel Quartus software on an Intel Stratix V as a target platform.

The process of simulating and assessing the maximum operation frequency has been carried out for a varying number of Rotators. This gives us the chance to select the desired trade-off between throughput θ and resources in Adaptive Logic Modules (ALMs) and number of registers. Results are collected in Table I, where the on-chip memory size is not reported, as it is roughly constant at about 2.19 Mbit. As expected, both logic elements and on-chip registers scale linearly with the number of Rotators. For all number of rotators between 8 and 48, the latency introduced by the Syndrome Processor is roughly constant and bound to 48k clock cycles. At the operating frequency of 125 MHz this corresponds to a processing time of about 0.38 ms. From throughput reduction

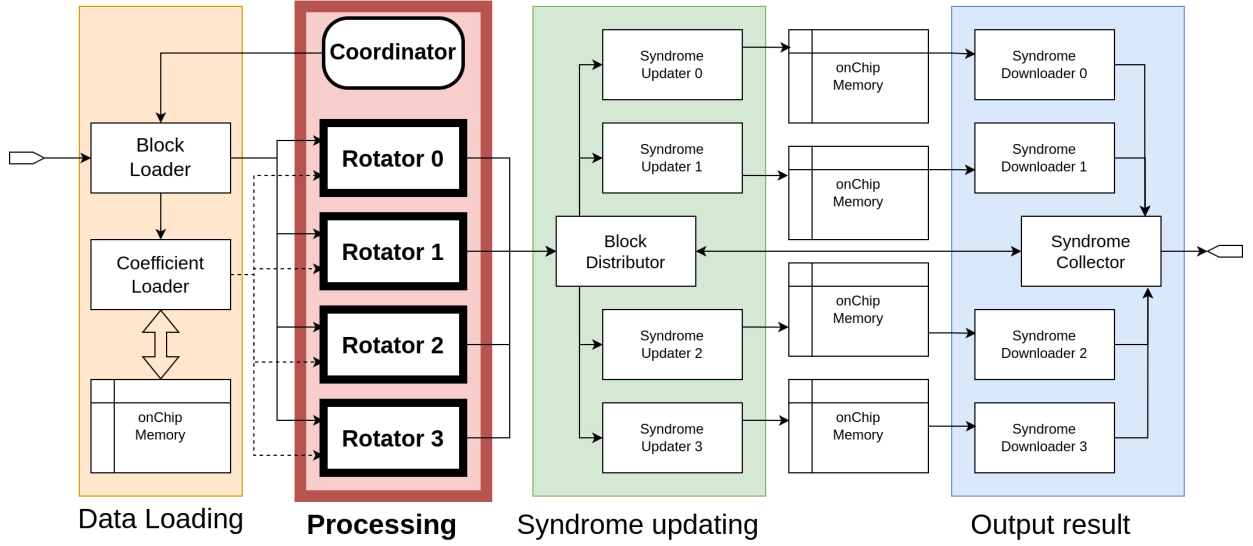


Fig. 2. Modules of the Syndrome Processor

and latency rise moving from 48 to 64 Rotators, we deduce that contention of shared resources is present.

An optimal throughput of 3.55 Gbit/s is obtained with 24 Rotators: with larger numbers the Syndrome Processor does not exhibit better throughput, while it consumes more resources. The maximum input rate that the Syndrome Processor is able to accept is 512 bits every 4 clock cycles, i.e. 16 Gbit/s at 125 MHz. Consequently the throughput is upper bounded by $(1 - R) \cdot 16 \lesssim 16$ Gbit/s. Despite the wide margin of improvement, the implementation of decoders usually exhibit much lower throughput. So, this encoder implementation is far from being the limiting factor of the system.

TABLE I
SUMMARY OF THROUGHPUT AND HARDWARE RESOURCES.

Rotators	T_c [cycles]	θ [Gbit/s] @ 125 MHz	ALMs	Registers
4	73k	1.89	11.6k	28.8k
8	46k	3.2	14.2k	35.7k
16	48k	3.11	20.2k	50.8k
24	42k	3.55	25.9k	66.2k
32	43k	3.53	31.5k	78.1k
48	43k	3.53	39.9k	106.0k
64	62k	2.76	50.8k	134.7k

IV. CONCLUSION

In this work we report the outcome of the design of the Syndrome Processor. We assessed linear scalability of both logic resources and registers for the FPGA implementation of the module. The latency introduced by syndrome processing is limited to approximately 0.38 ms. The achievable syndrome calculation throughput is at least 3.55 Gbit/s, but can be improved by tackling resource contention.

The Syndrome Processor can be integrated with a Mapper module, which is in charge of performing the core processing of Multi-Dimensional Reconciliation, i.e., joint mapping of

Gaussian distributed data (obtained from coherent photo-detection) and the input word into information-free coefficient.

ACKNOWLEDGEMENT

This work was partially carried out within the Department of Excellence in AI and Robotics of Scuola Superiore Sant'Anna, and was supported by the SMARTQKD scholarship, funded by Sma-RTy and CNR-IEIIT.

REFERENCES

- [1] P. W. Shor, "Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer," *SIAM Rev.*, vol. 41, no. 2, pp. 303–332, Jan. 1999, publisher: Society for Industrial and Applied Mathematics.
- [2] A. Leverrier and P. Grangier, "Simple proof that Gaussian attacks are optimal among collective attacks against continuous-variable quantum key distribution with a Gaussian modulation," *Phys. Rev. A*, vol. 81, no. 6, p. 062314, Jun. 2010, publisher: American Physical Society.
- [3] P. Jouguet, S. Kunz-Jacques, A. Leverrier, P. Grangier, and E. Diamanti, "Experimental demonstration of long-distance continuous-variable quantum key distribution," *Nature Photon.*, vol. 7, no. 5, pp. 378–381, May 2013.
- [4] H. Li and Y. Pang, "FPGA-Accelerated Quantum Computing Emulation and Quantum Key Distillation," *IEEE Micro*, vol. 41, no. 4, pp. 49–57, Jul. 2021.
- [5] M. Milicevic, C. Feng, L. M. Zhang, and P. G. Gulak, "Quasi-cyclic multi-edge LDPC codes for long-distance quantum cryptography," *npj Quantum Inf.*, vol. 4, no. 1, pp. 1–9, Apr. 2018, number: 1 Publisher: Nature Publishing Group.
- [6] Q. Lu, Z. Lu, H. Yang, S. Yang, and Y. Li, "FPGA-Based Implementation of Multidimensional Reconciliation Encoding in Quantum Key Distribution," *Entropy*, vol. 25, no. 1, p. 80, Dec. 2022.
- [7] K. Cui, J. Wang, H.-F. Zhang, C.-L. Luo, G. Jin, and T.-Y. Chen, "A Real-Time Design Based on FPGA for Expedient Error Reconciliation in QKD System," *IEEE Transactions on Information Forensics and Security*, vol. 8, no. 1, pp. 184–190, Jan. 2013.
- [8] S.-S. Yang, J.-Q. Liu, Z.-G. Lu, Z.-L. Bai, X.-Y. Wang, and Y.-M. Li, "An FPGA-Based LDPC Decoder With Ultra-Long Codes for Continuous-Variable Quantum Key Distribution," *IEEE Access*, vol. 9, pp. 47 687–47 697, 2021.